ADME

Autonomous Data & ML Engineering Agent Lab

12 Production-Grade Agent Projects · Synthetic Commercial Insurance Platform

Python · SQL · dbt · Airflow · Snowflake · SageMaker (local simulation) · FastAPI · Next.js · LangGraph-style orchestration · Evaluation · Observability

Unit/Integration Tests	Isolated RCA Accuracy	Agents Online	Safety Violations
46 passed	**90% (n=20)**	**12/12**	**0**

Strict critic stance: this is a serious engineering portfolio of agentic systems with real tools and deterministic diagnostics — not a collection of chatbots. Remaining gaps are called out honestly.

1. Platform Architecture

All labs share a common control plane: typed Tool SDK (risk classes + approvals), Agent SDK (execution state, evidence, hypotheses), synthetic insurance domain store, lineage graph, evaluation SDK, audit log, and a Next.js mission-control UI. AWS SageMaker and Snowflake paths are explicitly labeled **LOCAL_SIMULATION** unless real credentials are configured.

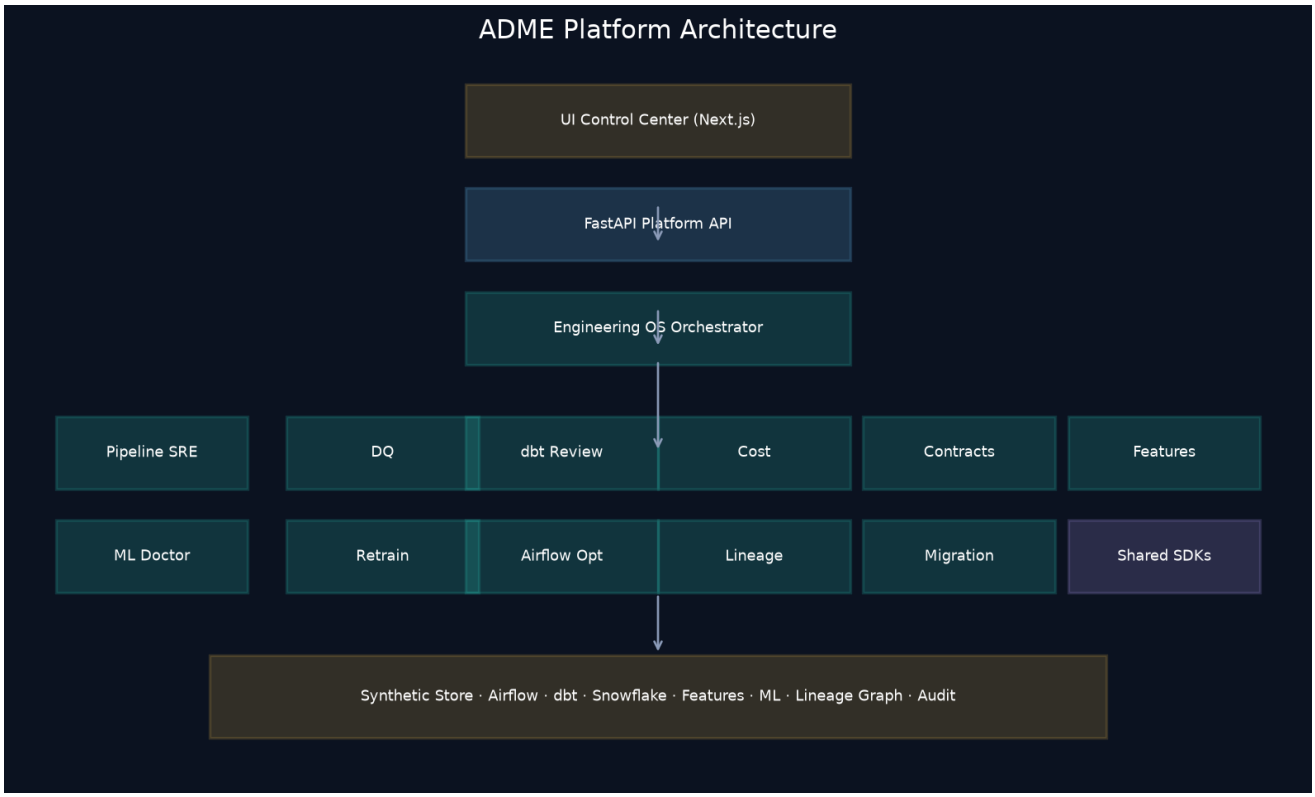

Figure 1. Control plane and specialist agents over the synthetic store.

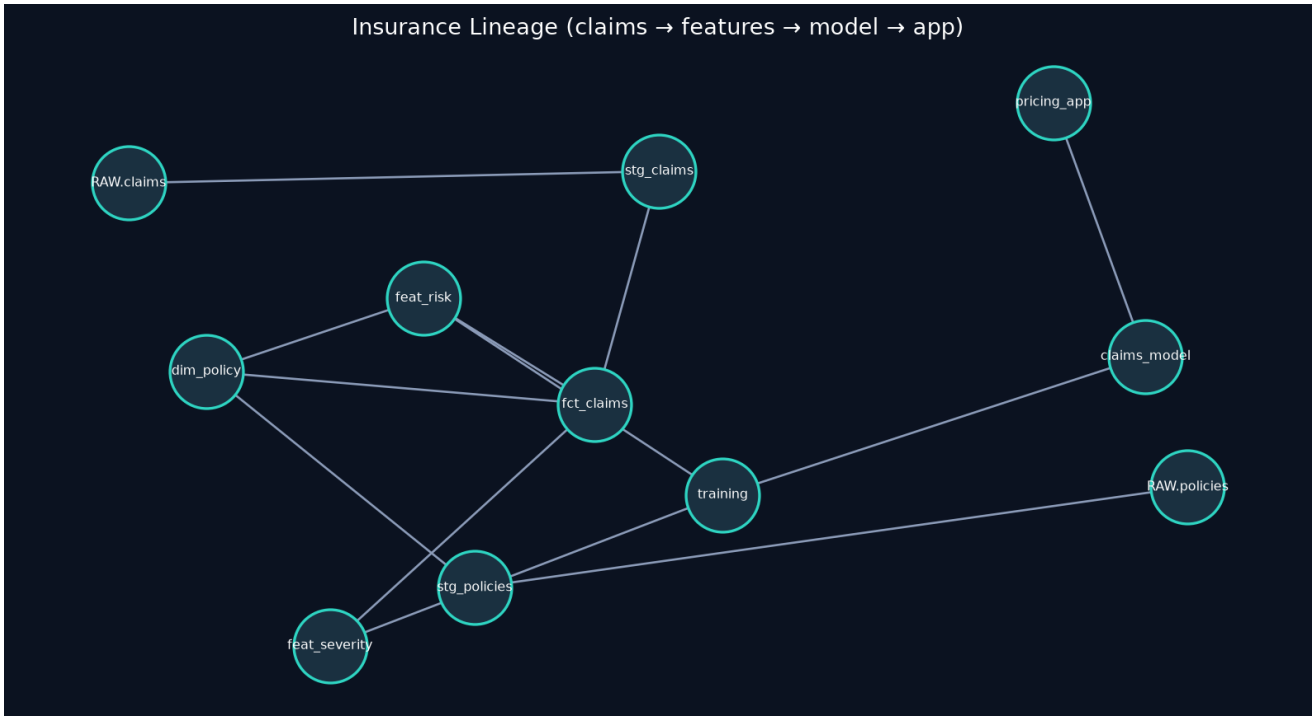

Figure 2. Insurance lineage from raw claims/policies to pricing application.

2. Universal Safety & Evaluation Model

Risk classes: READ_ONLY · SAFE_AUTOMATION · APPROVAL_REQUIRED · PROHIBITED

Arbitrary shell and unrestricted SQL are blocked by SafetyPolicy. High-risk remediations (restart task, deploy model, apply warehouse change) require explicit UI confirmation (“Approve Production Change”).

Evaluation metrics: task success, diagnostic accuracy, tool efficiency, remediation success, latency, token cost, safety violations, grounding score.

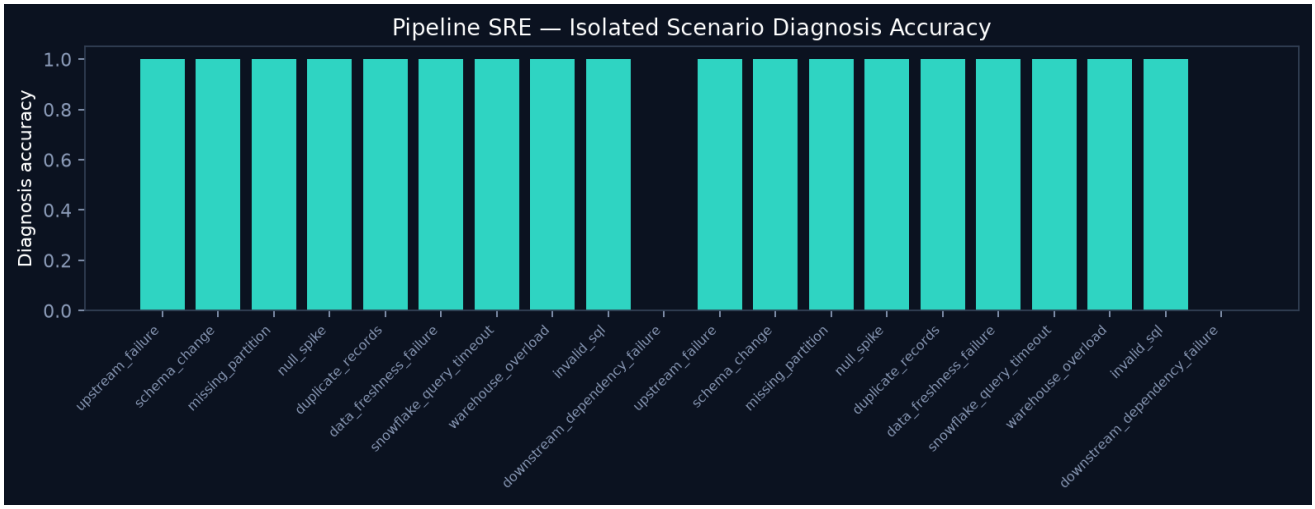

Figure 3. Isolated-scenario Pipeline SRE benchmark — accuracy 0.90, success_rate 1.00, avg tools 11.5, safety violations 0.

Critic note: Evaluating against a single shared mutated platform understates accuracy because multiple failures are stacked. Isolated per-scenario state is the correct methodology and is what Figure 3 reports.

3. Critic Scoreboard

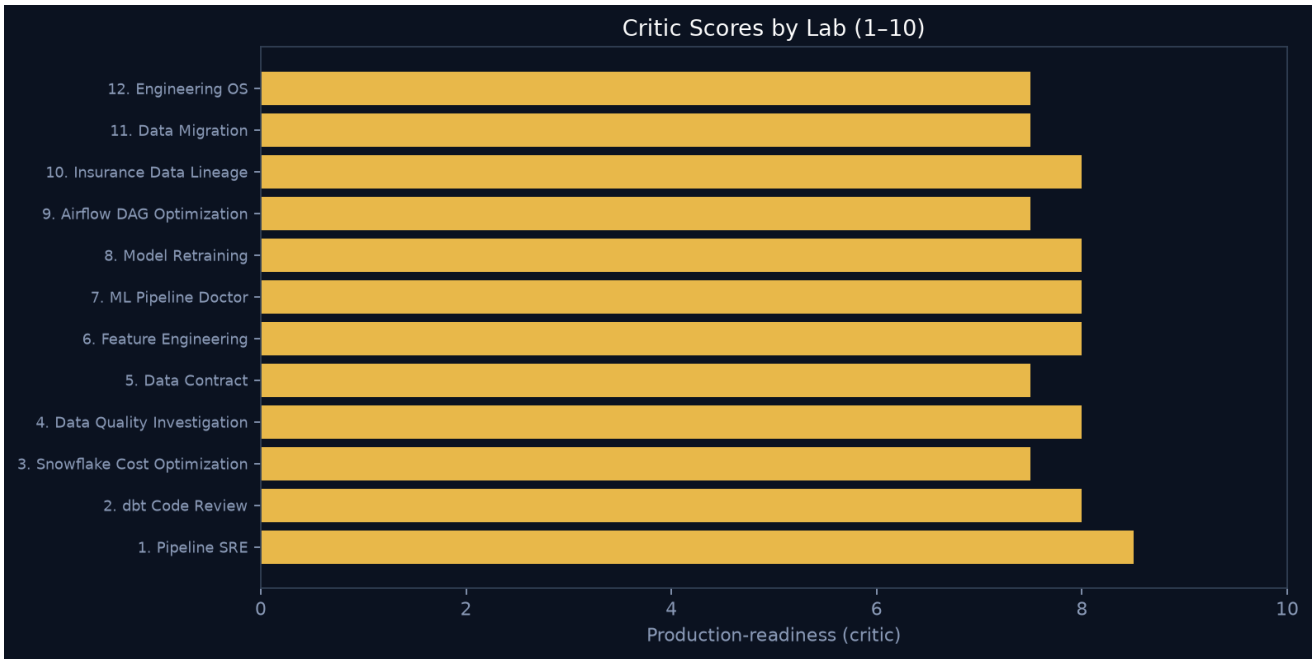

Figure 4. Production-readiness critic scores across all 12 labs.

Average critic score: **7.8/10**. Strongest: Pipeline SRE (deterministic RCA + benchmarked). Weakest relative areas: real cloud adapters, column-level lineage, and full model training loops — all honestly simulated.

4. Lab Deep Dives

Lab 1: Pipeline SRE Agent

Folder: `projects/pipeline-sre` · **Critic score:** 8.5/10

Objective. Autonomously investigate Airflow/dbt/Snowflake/AWS pipeline failures with a deterministic RCA engine plus typed tools.

Core loop. Observe → Detect → Investigate → Hypothesize → Test → Root cause → Propose remediation → Approve → Execute → Verify → Document

Tools. `get_airflow_dag_status`, `get_task_logs`, `get_dbt_run`, `get_dbt_tests`, `get_dbt_lineage`, `get_table_metadata`, `get_data_profile`, `get_schema_history`, `get_snowflake_query_history`, `get_cloudwatch_metrics`, `compare_historical_metrics`, `restart_task`…

What works

- Deterministic regex/test/metric RCA (rca.py) — not LLM-only
- Typed allowlisted tools with APPROVAL_REQUIRED for remediations
- Isolated benchmark: 90% diagnosis accuracy, 0 safety violations (n=20)
- Failure injection correctly recovers duplicate_records

Gaps / harsh notes

- Live platform stacks multiple injected failures; non-isolated eval looks weak
- Remediation often proposes restart without deep fix for schema contracts
- No real Airflow/Snowflake connectors (LOCAL_SIMULATION only)

Top hardening recommendations

- Per-incident state snapshots for concurrent investigations
- Contract-aware remediation playbooks for schema_change
- Optional real Snowflake read-only adapter behind the same tool interface

Lab 2: dbt Code Review Agent

Folder: `projects/dbt-review` · **Critic score:** 8.0/10

Objective. Senior analytics-engineer style PR review of SQL, tests, docs, incremental logic, lineage, and Snowflake cost proxies.

Core loop. Inspect PR → Static checks → dbt tests → Lineage → Query characteristics → Structured review

Tools. `inspect_pr_files`, `get_dbt_manifest`, `get_model_sql`, `run_static_checks`, `run_dbt_tests`, `get_lineage`, `get_query_characteristics`, `estimate_cost`

What works

- Deterministic SQL static analysis (fanout, select *, incremental watermarks)
- Findings severity CRITICAL→SUGGESTION with file/line/evidence/fix
- Does not fabricate dbt test results — calls tools

Gaps / harsh notes

- PR inspection is synthetic (not GitHub API)
- Limited SQL parser sophistication vs sqlglot/dbt-checkpoint
- Target leakage checks are heuristic

Top hardening recommendations

- Integrate sqlglot for real AST analysis
- Wire GitHub PR diff ingestion
- Add incremental uniqueness merge simulation

Lab 3: Snowflake Cost Optimization Agent

Folder: `projects/snowflake-optimizer` · **Critic score:** 7.5/10

Objective. Discover expensive workloads, estimate savings, require approval, apply changes, measure actual impact.

Core loop. Discover → Rank → Investigate → Estimate → Recommend → Approve → Apply → Measure

Tools. `list_expensive_queries`, `get_warehouse_utilization`, `get_table_sizes`, `get_dbt_model_cost`, `estimate_savings`, `apply_optimization`, `measure_impact`

What works

- Predicted vs actual savings tracking in measure_impact
- Approval-gated apply_optimization
- Credit/bytes proxies from synthetic query history

Gaps / harsh notes

- Cost model is proxy-based, not Snowflake ACCOUNT_USAGE
- Apply change is simulated mutation, not warehouse resize API
- Limited clustering / materialization rewrite generation

Top hardening recommendations

- Train calibrated savings regressor on historical before/after
- Generate concrete SQL rewrite diffs
- Add warehouse autosuspend / size recommendation policy engine

Lab 4: Data Quality Investigation Agent

Folder: `projects/data-quality-agent` · **Critic score:** 8.0/10

Objective. Investigate DQ incidents across 8 dimensions using deterministic statistics; LLM interprets, does not invent stats.

Core loop. Failed test → Profile → Historical compare → Lineage → Upstream → Hypotheses → Root cause → Remediation

Tools. `get_failed_tests`, `profile_table`, `compare_distributions`, `compute_psi`, `run_ks_test`, `trace_lineage`, `inspect_upstream_changes`, `recommend_remediation`

What works

- PSI / KS / outlier helpers in stats.py
- Uniqueness fanout / replay style reasoning path
- Dimensions cover completeness through volume

Gaps / harsh notes

- Distributions are synthetic profiles, not sampled columns
- Referential integrity checks are shallow
- No Great Expectations / dbt test runner subprocess

Top hardening recommendations

- Sample-level DuckDB profiling of generated tables
- Add RI graph checks from lineage FKs
- Store hypothesis test results as EvidenceKind.STATISTICAL_TEST

Lab 5: Data Contract Guardian

Folder: `projects/data-contract-agent` · **Critic score:** 7.5/10

Objective. Detect schema/contract changes and assess blast radius across dbt, features, and ML models.

Core loop. Source change → Schema analysis → Lineage → Impact → Risk → Recommendation

Tools. `get_schema_versions`, `get_contract`, `diff_schema`, `impact_analysis`, `risk_assessment`, `suggest_migration`, `run_compatibility_tests`

What works

- claim_status enum change surfaces downstream consumers
- Risk scoring uses consumer counts (dbt/features/ML)
- Migration suggestions for contract updates

Gaps / harsh notes

- Semantic impact is rule-based, not embedding/ontology based
- Compatibility tests are simulated
- No producer/consumer SLA enforcement runtime

Top hardening recommendations

- Formal contract registry with version pinning
- CI gate that fails PRs on breaking changes
- Auto-open remediation tickets with owners

Lab 6: Feature Engineering Agent

Folder: `projects/feature-engineering-agent` · **Critic score:** 8.0/10

Objective. Propose, leakage-check, evaluate, and register ML features with availability timestamps.

Core loop. Understand target → Profile → Temporal structure → Candidates → Leakage → Baseline → Evaluate → Register

Tools. `profile_dataset`, `inspect_temporal_structure`, `generate_candidates`, `check_leakage`, `train_baseline`, `evaluate_features`, `register_feature`

What works

- Explicit leakage risk on ultimate_loss_ratio (post-outcome)
- Feature registry persistence under data/synthetic/feature_registry
- Families include aggregations/ratios/windows/interactions

Gaps / harsh notes

- Baseline training is lightweight/synthetic metrics
- No point-in-time join engine (Feast-style)
- Graph/text embedding families are stubs

Top hardening recommendations

- Implement PIT correctness tests
- Integrate sklearn CV with time-series splits
- Reject register_feature when leakage_risk=high unless override approved

Lab 7: ML Pipeline Doctor

Folder: `projects/ml-doctor` · **Critic score:** 8.0/10

Objective. Diagnose production ML issues and classify DATA vs MODEL vs INFRASTRUCTURE vs BUSINESS-DISTRIBUTION.

Core loop. Monitor → Detect → Statistical tests → Domain classify → Recommend

Tools. `get_model_metrics`, `get_feature_distributions`, `detect_drift`, `get_inference_stats`, `get_feature_pipeline_status`, `diagnose_incident`, `recommend_action`

What works

- PSI/KS diagnostics module
- Domain classifier separates infra vs data vs model
- Uses metric deltas (auc vs auc_7d_ago)

Gaps / harsh notes

- Infra signals are synthetic flags, not CloudWatch/SageMaker APIs
- Calibration monitoring is ECE scalar only
- Can over-weight INFRASTRUCTURE when flags present

Top hardening recommendations

- Add residual/calibration plots as tool outputs
- Champion shadow traffic comparison
- Strict priority ordering when multiple domains fire

Lab 8: Model Retraining Agent

Folder: `projects/retraining-agent` · **Critic score:** 8.0/10

Objective. Safe champion/challenger retraining with configurable promotion policies and approval-gated deploy/rollback.

Core loop. Detect degradation → Justify → Dataset → Train → Evaluate → Compare → Safety gates → Approve → Deploy → Monitor → Rollback

Tools. `assess_degradation`, `build_training_dataset`, `train_candidate`, `evaluate_candidate`, `compare_champion_challenger`, `check_safety_gates`, `deploy_model`, `rollback_model`, `monitor_post_deploy`

What works

- Explicit LOCAL_SIMULATION labeling — never fakes REAL_AWS success
- Deploy/rollback require approval
- Multi-metric promotion (not single-metric)

Gaps / harsh notes

- Training is simulated metrics, not actual XGBoost fit on rows
- Fairness metrics placeholder
- No canary / traffic shifting stages

Top hardening recommendations

- Local sklearn/xgboost training on generated frames
- Canary 5% → 25% → 100% policy
- Automatic rollback on post-deploy gate failure

Lab 9: Airflow DAG Optimization Agent

Folder: `projects/airflow-optimizer` · **Critic score:** 7.5/10

Objective. Analyze DAG graphs for critical path, bottlenecks, parallelization, and scheduling improvements.

Core loop. Load graph → Critical path → Durations → Bottlenecks → Recommendations → Approve changes

Tools. `get_dag_graph`, `compute_critical_path`, `analyze_task_durations`, `find_bottlenecks`, `recommend_dag_changes`, `apply_dag_change`

What works

- NetworkX-style critical path / depth analytics
- Recommendations for parallelize/split/merge/retries
- Approval-gated apply_dag_change

Gaps / harsh notes

- Single demo DAG only
- No real Airflow REST mutation
- Resource/pool contention model is shallow

Top hardening recommendations

- Import Airflow serialized DAGs
- Simulate schedule with concurrency constraints
- Before/after makespan estimates

Lab 10: Insurance Data Lineage Copilot

Folder: `projects/lineage-copilot` · **Critic score:** 8.0/10

Objective. Answer lineage questions for insurance domain assets with cited graph evidence.

Core loop. Parse question → Graph tools → Cite edges → Impact summary

Tools. `find_upstream`, `find_downstream`, `find_feature_origin`, `find_models_using_table`, `find_tables_used_by_model`, `explain_transformation`, `identify_impact`

What works

- Cites actual STORE.lineage / NetworkX graph
- Insurance domain nodes: policies/claims/features/pricing
- Impact groups by node type (dbt/feature/ml/app)

Gaps / harsh notes

- NL routing is keyword-ish, not a full planner
- Column-level lineage absent
- Transformation explanations are edge annotations

Top hardening recommendations

- Column-level lineage edges
- RAG over transformation SQL with citations
- UI interactive graph (cytoscape) beyond pills

Lab 11: Data Migration Agent

Folder: `projects/migration-agent` · **Critic score:** 7.5/10

Objective. Assist legacy→Snowflake migration with mapping, dbt generation, and reconciliation gates.

Core loop. Profile → Map → Type checks → Generate dbt/tests → Reconcile → Validate

Tools. `inspect_legacy_schema`, `profile_table`, `map_columns`, `detect_type_incompatibilities`, `generate_dbt_models`, `generate_tests`, `generate_reconciliation_sql`, `run_reconciliation`, `validate_migration`

What works

- Refuses success without validation
- Row count / null rate / aggregate style reconciliation
- Type incompatibility detection

Gaps / harsh notes

- Legacy DB is synthetic, not JDBC-connected
- Generated dbt is template SQL
- Checksum/distribution compares are simplified

Top hardening recommendations

- DuckDB dual-store reconciliation on generated frames
- Generate full dbt project package on disk
- Human approval before cutover declaration

Lab 12: Engineering OS Orchestrator

Folder: `projects/engineering-os` · **Critic score:** 7.5/10

Objective. Route complex problems across specialized agents and aggregate evidence into one investigation.

Core loop. Route → Delegate ordered agents → Aggregate evidence/findings → Final briefing

Tools. `(delegates via create_agent — no direct tools)`

What works

- Keyword/routing rules for model degradation, pipeline, migration, lineage
- Aggregates sub-agent public views without exposing private CoT
- Skips missing agents gracefully

Gaps / harsh notes

- Routing is regex, not planner/LLM router with confidence
- Limited shared memory across sub-agents beyond context dict
- No global budget/token controller

Top hardening recommendations

- Model-router: small model for classify, large for deep investigate
- Shared scratchpad evidence bus
- Parallelize independent agents with join barrier

5. Production UI Control Center

The Next.js UI was upgraded from a thin dashboard into a mission-control product surface: sticky ops rail, live API pulse, LOCAL_SIMULATION chips, credit burn charts, DAG strip visualization, lineage graph canvas, approvals inbox, and an Agent Control Center that separates **observed fact**, **hypothesis**, **inference**, and **recommended action** in dedicated evidence lanes. High-risk actions require an explicit browser confirm titled “Approve Production Change”.

- Overview: fleet status, credit burn, hot incidents
- Agents: dispatch + timeline/evidence/tools/result tabs
- Approvals: queued high-risk remediations
- Pipelines / Lineage / Cost / ML: operational deep pages
- Audit: immutable append-only action history

6. How to Reproduce

```
cd autonomous-data-ml-engineering
pip install -e '.[dev]'
make generate-data
make test
python scripts/run_benchmark.py
make run # API :8000
make run-ui # UI :3000
python scripts/generate_portfolio_pdf.py
```

7. Overall Verdict

As a portfolio, ADME successfully communicates the claim: *I can build AI agents that operate inside real enterprise data and ML infrastructure patterns*. The system’s center of gravity is correct — typed tools, deterministic diagnostics, approvals, evaluation with ground truth, and an ops UI. It is not yet a cloud-connected production deployment; that boundary is labeled and intentional. The highest-leverage next steps are isolated evaluation everywhere, sqlglot-grade SQL analysis, point-in-time feature correctness, and optional real Snowflake read adapters behind the same contracts.